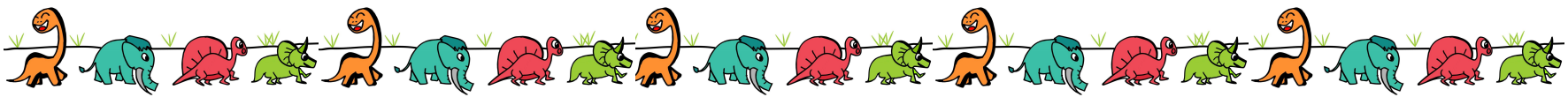
Compiler Optimizations

Lio Wright



Introduction to Compiler Optimizations

Compiler Optimizations: changes made to source code during compile time that improve performance while preserving behavior

Simple example: constant folding, i.e., evaluating constants at compile time instead of runtime. (This is distinct from constant propagation)



Goals of Compiler Optimization

- Avoid redundancy
- Reduce computations
- Reduce jumps (branch-free code)
- Improve locality of reference*
- Parallelize
- Strength reduction*

Vocab: Locality of Reference

- It is faster to retrieve memory that is in the cache vs main memory
- Memory that is 1) used frequently or 2) needed soon should be stored in the cache to improve performance
- The CPU tries to predict what it will need soon & store it in the cache
    - Lots of conditionals can reduce the accuracy of these predictions.

Vocab: Strength Reduction

- Replacing complex/expensive operations with simpler/cheaper ones.

Simple example: bitwise addition is cheaper than regular addition

Flags

Introduction to Flags in Compiler Optimization

How much do you want the compiler to optimize?

- As a general rule, more optimizations results in a slower compile time and faster runtime (and vice versa.)
  - Some optimizations are faster at compile time. Some actually slow down the program at runtime*.
- The optimizations associated with a flag can vary by system.
- A single optimization or a group can be explicitly specified when compiling.

```
$gcc -O2 hello.c
```

Most Common Flags

- -O0: Extremely minimal optimization. Default if not specified.
  - Reduce compile time and produces expected results when debugging
  - -falign-loops, -finline-functions-called-once, -fmove-loop-invariants are still used
- -O1: limited optimization, avoiding substantial increase to compile time
  - Useful for large programs that may take awhile to compile or programs with many branches. Not useful for programs that are mostly loops
- -O2: O1 + nearly all optimizations without a space-speed tradeoff
  - General purpose, web servers, large codebases
- -O3: Aggressive optimization, favoring runtime over compile time.
  - Useful for loop-heavy code, video processing, or large datasets (scientific programming)

Trade Offs

It varies by program, but these tradeoffs are broadly true.

Flag	Runtime	Compile Time	Code Size	Memory Usage
-O0	Slow	Fast	Larger	Less
-O1	Fast	Slower	Smaller	More
-O2	Faster	Slower		More
-O3	Faster	Slowest		More

Other Flags

- -Ofast: extremely aggressive optimizing, to the point that it may change program behavior
  - Ignores standard compliance (e.g. floating point shortcuts)
- -Og: optimizations that don’t interfere with debugging
  - This is effectively “improved” -O0
- -Os: optimizes for code size in addition to speed. Enables all -O2 optimizations that do not increase code size
- There are some platform specific optimizations as well

Optimizations Must Suit the Program

- O3 is more *aggressive*. It is not always faster.
  - E.g. it may produce significantly larger code size in the resulting compiled binary, which is slower for the CPU to chew through
- When in doubt, you can compile with both O2 and O3 and test which is faster

Technique 1: Dead Code Elimination (DCE)

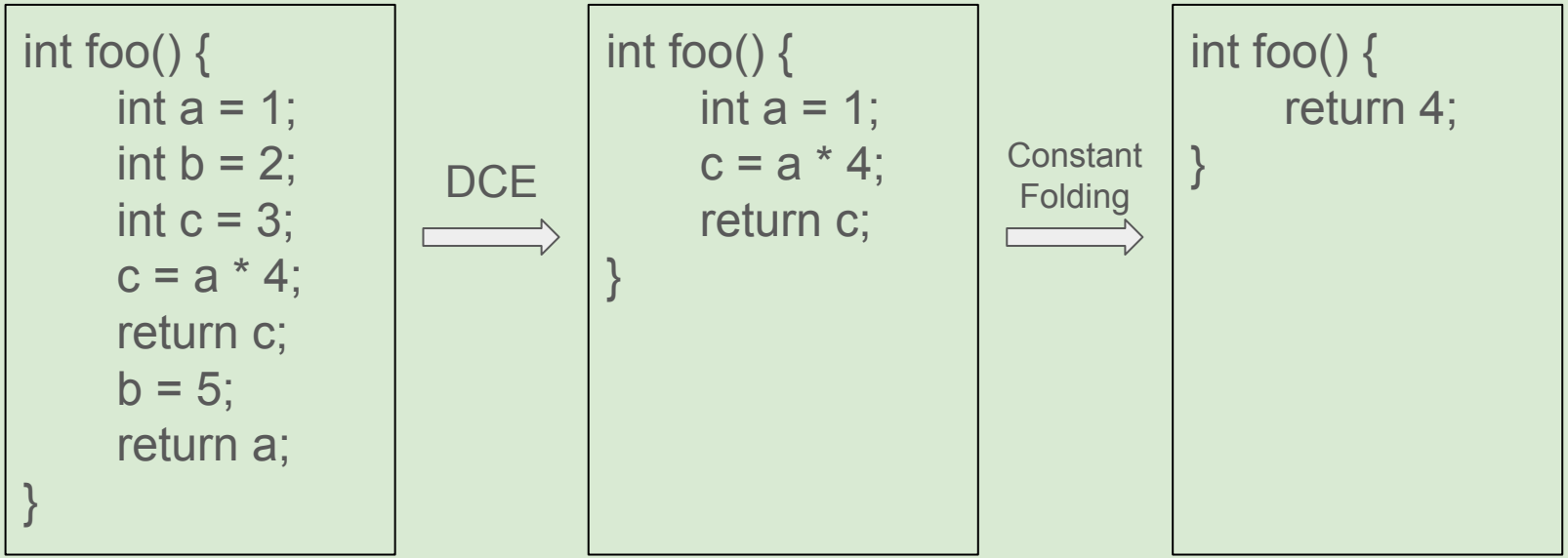
Introduction to DCE

- Removes code that does not affect behavior
  - Unreachable code
  - Dead variables: variables that are updated/assigned but that value is never used
  - Unused functions
- Improves performance by reducing the size of the code and skipping unnecessary operations at runtime.
- Improves readability
- Dead code is often created by other optimizations!

Static Analysis

- Static analysis: before running a program, source code is checked. This is how the compiler finds dead code
- Parsing and AST generation: Source code is read, divided, and traced.
  - Abstract search tree (AST): maps structure of code
- Control Flow Analysis: Execution paths are traced. If any code is found unreachable, it’s dead.
- Data Flow Analysis/Liveness Analysis: Tracks writes and reads. If an assigned value is written but never read, it’s dead.

Example of DCE



Potential Errors

- It can mistake things that would happen at runtime (eg reflection)
- It is possible to have a seemingly dead section of code that actually does affect behavior, which is called a side effect (eg modifying global variables or importing a css style sheet)

Technique 2: Loop Optimizations

Introduction to Loop Optimizations

- Improves runtime and reduces overhead by transforming loops.
- Improves cache performance and utilizes parallel-performance capabilities
- Remember -O3?

Some Quick Examples of Loop Operations

- Fission: a loop is split into several smaller loops, improving locality of reference
- Fusion: combining two independent loops that run the same number of times
- Loop-Invariant Code Motion: If every iteration of a loop has common code, that code is pulled outside of the loop.
  - Requires inversion*
- Parallelization: Restructuring loops for systems with multiple processors such that the additional processors are not idle
  - Does not work for sequential items
- Loop Unrolling: Rewriting a loop as a sequence of repeated instructions
  - Reduces “computational overhead” (pointer arithmetic, evaluating loop conditions, etc.)
  - This is a tradeoff between speed and binary size

Inversion

Reducing pipeline stall by switching while loops to do-while loops

```
int i = 0;
arr[20];
while (i < 20) {
        arr[i] = i;
        i++;
}
```



```
int i = 0;
arr[20];
if (i < 20) {
        do {
                arr[i] = i;
                i++;
        } while (i < 20);
}
```

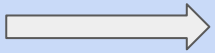
Vectorization

If a loop applies the same operation to consecutive elements, several iterations can be combined into one iteration.

Converts scalar code into SIMD code

Source code

```
void add(float *a, const float *b,
const float *c, int n) {
    for (int i = 0; i < n; i++) {
        a[i] = b[i] + c[i];
    }
}
```



Conceptual result of auto-vectorization

```
int i = 0;
for (; i + 8 <= n; i += 8) {
    // body
}
for (; i < n; i++) {        //leftover
    a[i] = b[i] + c[i];
}
```

Vectorization Conditions

- Loops are independent
- Simple flow (branches could interfere with vectorization)
- No aliasing problems

Vectorization can actually slow down simple loops because of the time taken to process the overhead.

Summary

- Compiler optimizations can improve performance in several different categories.
- This is all about selecting the best tradeoffs for your code.
- Understanding how the compiler works is important for running, testing, and debugging programs.